1.2 Przegląd modeli: GPT-4o, Claude 3.5 Sonnet i modele lokalne (Llama 3)

Krajobraz modeli językowych — orientacja w terenie

Świat dużych modeli językowych (LLM) zmienia się w zawrotnym tempie — nowe wersje i nowi gracze pojawiają się co kilka miesięcy.

Dla AI developera kluczowe jest nie tyle śledzenie każdego benchmarku, co **rozumienie osi decyzyjnych**:

- Kiedy sięgnąć po GPT-4o?
- Kiedy wybrać Claude 3.5 Sonnet?
- Kiedy uruchomić model lokalnie na własnym sprzęcie?

Decyzja zależy od kontekstu projektu — chatbot dla korporacji wymaga innych modeli niż badacz eksperymentujący na laptopie.

Osie decyzyjne przy wyborze modelu

Modele językowe można klasyfikować według czterech kluczowych kryteriów:

- **Dostęp:** API w chmurze (OpenAI, Anthropic, Google) vs. modele open-weight uruchamiane lokalnie (Llama, Mistral, Phi)
- **Rozmiar:** mierzony liczbą parametrów — od kilku miliardów (7B, 8B) do setek miliardów i powyżej
- **Specjalizacja:** modele ogólne vs. wyspecjalizowane (kod, matematyka, medycyna)
- **Koszt licencji:** modele komercyjne z zamkniętymi wagami vs. modele open-weight dopuszczające lokalne wdrożenia

Każda z tych osi ma bezpośrednie przełożenie na wybór modelu w projekcie.

GPT-4o — multimodalny model flagowy OpenAI

GPT-4o (gdzie „o" oznacza *omni*) to model OpenAI łączący rozumienie tekstu, obrazów i dźwięku w jednym modelu end-to-end.

Mocne strony GPT-4o:

- Wyjątkowo dobre rozumienie poleceń złożonych i wieloetapowych zadań
- Natywna obsługa obrazów — analiza zrzutów ekranu, diagramów i kodu w formie zdjęć
- Bardzo duże okno kontekstowe: **128 000 tokenów**
- Doskonała integracja z ekosystemem OpenAI: Assistants API, function calling, batch processing
- Wbudowane narzędzia: generowanie kodu z uruchamianiem w sandboxie oraz wyszukiwanie w sieci

GPT-4o — ograniczenia i kiedy wybrać

Słabsze strony i ograniczenia:

- Model z zamkniętymi wagami — brak możliwości lokalnego uruchomienia ani fine-tuningu poza API
- Koszty wywołań mogą rosnąć szybko przy dużej skali (choć koszt tokenów systematycznie spada)
- Brak pełnej transparentności co do architektury i danych treningowych

Kiedy wybrać GPT-4o:

- Produkcyjne aplikacje wymagające wysokiej niezawodności
- Złożone zadania rozumowania wielokrokowego
- Projekty wykorzystujące multimodalność (obraz + tekst + dźwięk)
- Integracja z istniejącym ekosystemem OpenAI

Claude 3.5 Sonnet — model Anthropic

Claude 3.5 Sonnet to model firmy Anthropic, wyróżniający się właściwościami istotnymi dla zastosowań biznesowych i deweloperskich.

Kluczowe cechy Claude 3.5 Sonnet:

- **Bardzo duże okno kontekstowe** — do **200 000 tokenów**, umożliwiające analizę całych repozytoriów kodu lub długich dokumentów prawnych w jednym wywołaniu
- **Wyjątkowa jakość generowania kodu** — w benchmarku SWE-bench Claude 3.5 Sonnet osiągał najlepsze wyniki spośród komercyjnych modeli
- **Bardzo dobre rozumienie długich instrukcji** — model rzadziej „gubi" kontekst z początku długiego promptu
- Większa przewidywalność i mniejsza skłonność do halucynacji (wymyślania nieprawdziwych faktów)

Claude 3.5 Sonnet — Constitutional AI i filozofia Anthropic

Constitutional AI to technika trenowania opracowana przez Anthropic, w której model jest uczony zestawem zasad etycznych.

W praktyce oznacza to:

- Większą **przewidywalność** zachowania modelu w środowiskach produkcyjnych
- Mniejszą skłonność do generowania szkodliwych lub nieprawdziwych treści
- Model częściej odmówi wykonania ryzykownych poleceń — co bywa ograniczeniem, ale zwiększa bezpieczeństwo

Firma Anthropic kładzie duży nacisk na **bezpieczeństwo i zgodność** (*alignment*).

Kiedy wybrać Claude 3.5 Sonnet: Analiza dużych ilości kodu lub dokumentów, generowanie i refaktoryzacja kodu, projekty wymagające spójności i wysokiej jakości generowanego tekstu.

Llama 3 i modele open-weight — wolność i odpowiedzialność

Llama 3 to rodzina modeli open-weight opracowana przez Meta AI. Dostępne warianty: **8B, 70B i 405B** parametrów.

Najważniejsza cecha: modele uruchamiają się lokalnie. Oznacza to:

- **Prywatność danych** — żadne dane nie opuszczają własnej infrastruktury; kluczowe w sektorach regulated (zdrowie, finanse, prawo)
- **Brak opłat za wywołania** — koszt to infrastruktura (GPU/CPU), a nie subskrypcja API
- **Pełna kontrola** — możliwość fine-tuningu na własnych danych, modyfikacji system promptów, integracji z dowolną infrastrukturą
- **Niższe latencje** — przy odpowiednim sprzęcie czas odpowiedzi może być krótszy niż przy zdalnym API

Llama 3 — ograniczenia i narzędzia do uruchamiania

Ograniczenia modeli lokalnych:

- Wymagają GPU lub dużej ilości RAM: model 8B potrzebuje 8–16 GB VRAM; 70B wymaga konfiguracji wielokartowej lub kwantyzacji
- Jakość generowania jest zazwyczaj niższa niż flagowych modeli komercyjnych (choć różnice maleją z każdą generacją)
- Zarządzanie infrastrukturą, aktualizacjami i bezpieczeństwem spoczywa na deweloperze

Narzędzia do uruchamiania modeli lokalnych:

- **Ollama** — najprostszy sposób na uruchomienie Llamy i innych modeli jedną komendą w terminalu
- **LM Studio** — graficzny interfejs z wbudowanym API kompatybilnym z OpenAI
- **vLLM** — wydajny serwer inferencji dla środowisk produkcyjnych

Porównanie modeli — tabela zestawcza

Cecha	GPT-4o	Claude 3.5 Sonnet	Llama 3 (70B)
Producent	OpenAI	Anthropic	Meta AI
Okno kontekstowe	128 000 tokenów	200 000 tokenów	8 192–131 072 tokenów
Dostęp	API (chmura)	API (chmura)	Lokalne / własna chmura
Prywatność danych	Wysyłane do OpenAI	Wysyłane do Anthropic	W pełni lokalna
Multimodalność	Tekst, obraz, dźwięk	Tekst, obraz	Tekst (warianty Vision)
Fine-tuning	Ograniczony (przez API)	Niedostępny	Pełny dostęp
Koszt użycia	Per token	Per token	Infrastruktura własna
Jakość kodu	Bardzo wysoka	Bardzo wysoka	Wysoka (70B+)
Licencja	Komercyjna zamknięta	Komercyjna zamknięta	Meta Llama License

Benchmarki — co mierzą i jak je czytać

Przy wyborze modelu deweloperzy często sięgają po wyniki benchmarków. Warto rozumieć, co mierzą najpopularniejsze z nich:

- **MMLU** (Massive Multitask Language Understanding) — wiedza ogólna i rozumowanie w 57 dziedzinach; dobry miernik „encyklopedyczności" modelu
- **HumanEval / MBPP** — rozwiązywanie zadań algorytmicznych; benchmark dla jakości generowania kodu
- **SWE-bench** — realne bugi z GitHuba do naprawienia; najbardziej praktyczny benchmark dla AI developera
- **GPQA** — pytania z fizyki, chemii i biologii na poziomie doktoratu; mierzy głębokie rozumowanie

Każdy benchmark mierzy inny aspekt modelu — żaden nie jest kompletną miarą użyteczności.

Benchmarki — dlaczego nie ufać im ślepo

Wyniki benchmarków mogą wprowadzać w błąd z kilku powodów:

- **Data contamination** — modele mogą być trenowane na zestawach testowych, co sztucznie zawyża wyniki
- Wyniki w benchmarkach **nie zawsze przekładają się** na rzeczywistą wydajność w konkretnym zadaniu (np. generowanie kodu w TypeScript vs. Python może się różnić)
- Liczy się całościowe doświadczenie: szybkość, koszt, niezawodność API, latencja — czego benchmarki nie mierzą

Rekomendacja praktyczna: Zamiast ufać tylko tabelom porównawczym, przetestuj model na własnym zestawie promptów reprezentatywnych dla projektu. Godzina testów własnoręcznych daje więcej informacji niż dziesiątki artykułów porównawczych.

Strategia wyboru modelu — model routing

W praktyce wiele projektów korzysta z **kombinacji modeli** dopasowanych do zadania. Podejście to zwane jest *model routing* lub *cascading*:

- **Szybkie, tanie zadania** (klasyfikacja, ekstrakcja danych, formatowanie) → małe modele lokalne lub `gpt-4o-mini`
- **Złożone rozumowanie, generowanie kodu** → GPT-4o lub Claude 3.5 Sonnet
- **Praca z dokumentami wrażliwymi** → Llama 3 uruchomiona lokalnie
- **Fine-tuning do wąskiej domeny** → otwarty model (Llama, Mistral) dotrenowany na własnych danych

Decyzja o wyborze modelu to kompromis między **jakością**, **kosztem**, **prywatnością** i **kontrolą**.

Podsumowanie — świadomy wybór modelu

Kluczowe wnioski z przeglądu modeli:

- **GPT-4o** — najlepsza integracja z ekosystemem OpenAI, multimodalność, duże okno kontekstowe (128K tokenów); wybór do produkcyjnych aplikacji wielokrokowych
- **Claude 3.5 Sonnet** — największe okno kontekstowe (200K tokenów), najwyższa jakość kodu (SWE-bench), Constitutional AI zapewniające przewidywalność
- **Llama 3** — pełna prywatność danych, brak kosztów per-token, możliwość fine-tuningu; wymaga własnej infrastruktury GPU
- Benchmarki są wskazówką, ale nie wyrocznią — **własne testy na reprezentatywnych promptach** są niezbędne
- Świadomy AI developer potrafi uzasadnić wybór modelu wobec klienta lub zespołu, biorąc pod uwagę jakość, koszt, prywatność i kontrolę